

Guía de Implementación

Visor DICOM — Portal Web Radiólogos

Instalación y configuración de Orthanc en Windows Server 2012 R2, IIS/ARR, OHIF e integración con la aplicación .NET 8

<https://appsintranet.esculapiosis.com/PortalImagenologia>

Proyecto	webImagenologia (WILSP1971/webImagenologia)
Servidor	Windows Server 2012 R2 · IIS · .NET 8
PACS	dcm4chee (DIMSE/WADO-URI) + Orthanc (DICOMweb)
Fecha	2026-08-07
Versión guía	1.0

Contenido

1. Contexto del proyecto
 2. Cómo expone los datos el PACS (hallazgos reales)
 3. Arquitectura de la solución (Plan A y Plan B)
 4. Instalación y configuración de Orthanc (Windows Server 2012 R2)
 5. IIS: reverse proxy ARR + URL Rewrite (sub-aplicación y hardening)
 6. OHIF: build y despliegue como aplicación estática
 7. Integración con la aplicación .NET 8 (gateway, token, auditoría)
 8. Seguridad y hardening
 9. Plan de pruebas end-to-end
 10. Bitácora de operación del enjambre (esta sesión)
 11. Solución de problemas y puesta en marcha de Orthanc
- Anexo A. Datos reales de referencia

1. Contexto del proyecto

Existe una aplicación web de Imagenología desarrollada en **C# / .NET 8**, publicada y funcional en **IIS** sobre **Windows Server 2012 R2**, disponible en:

<https://appsintranet.esculapiosis.com/PortalImagenologia>

La aplicación administra hoy la información clínica de estudios de imágenes diagnósticas. La data no sale de una BD directa: proviene de una **API externa ("Esculapio", IEsculapioApiClient)**. La autenticación es por cookie + IDataProtection, con roles **Administrador** y **Radiologo**.

Se requiere incorporar un **visor de imágenes diagnósticas** para estudios DICOM que residen en un servidor **PACS**. El estudio se localiza por **(a) Número de Caso/Cuenta** o **(b) Número de Identificación del paciente**.

El módulo donde se implementa el visor es **Portal Web Radiólogos**:

Controllers/PortalRadiologosController.cs, vista Views/PortalRadiologos/Index.cshtml, wwwroot/js/portalRadiologos.js.

Restricción dura

Todo lo nuevo del visor va en módulos/carpetas propios. **No se modifica el código operativo y funcional existente**. Nunca se suben al repositorio credenciales, TokenSecret ni archivos con datos de pacientes (PHI).

2. Cómo expone los datos el PACS (hallazgos reales)

Estos datos **no son suposiciones**: se extrajeron de la captura de red real del visor Oviyam en producción (endpoint DicomNodes.do y Echo.do de un archivo HAR).

Backends y orígenes

Existen dos backends PACS y cuatro orígenes lógicos (las pestañas de Oviyam):

Pestaña	Backend	Host	Puerto	AE Title	Recuperación
CAMPBELL	dcm4chee	172.16.10.100	11112	DCM4CHEE	WADO-URI :8080/wado (JPEG)
FUNDACIONCAMPBELL	dcm4chee	172.16.10.100	11112	DCM4CHEE	WADO-URI :8080/wado (JPEG)
SANTAMARTA	dcm4chee	172.16.50.100	11112	DCM4CHEE	WADO-URI :8080/wado (JPEG)
VISORIMAGENLOGICA	Orthanc	192.168.2.17	4242	ESCULAPIO_ORTHANC	WADO-URI :8080/wado (JPEG)

AE llamante de Oviyam: OVIYAM2 (listener 1025). C-ECHO real verificado y exitoso contra DCM4CHEE@172.16.10.100:11112 → **EchoSuccess**.

Las dos vías de exposición

- **DIMSE (DICOM clásico, TCP)**: C-FIND (búsqueda), C-MOVE / C-GET (recuperación). Puertos 11112 (dcm4chee) y 4242 (Orthanc).

- **WADO-URI (HTTP)**: recupera la imagen ya rasterizada —
`http://host:8080/wado?requestType=WADO&studyUID;=...&objectUID;=...` Es lo que usa Oviyam hoy, devolviendo JPEG.

Limitación clave

dcm4chee expone WADO-URI clásico (imagen JPEG/PNG por objeto) pero **no** DICOMweb REST moderno (QIDO-RS / WADO-RS / STOW-RS) que necesitan OHIF/Cornerstone para calidad diagnóstica (window/level dinámico, MPR). De ahí la estrategia del gateway Orthanc.

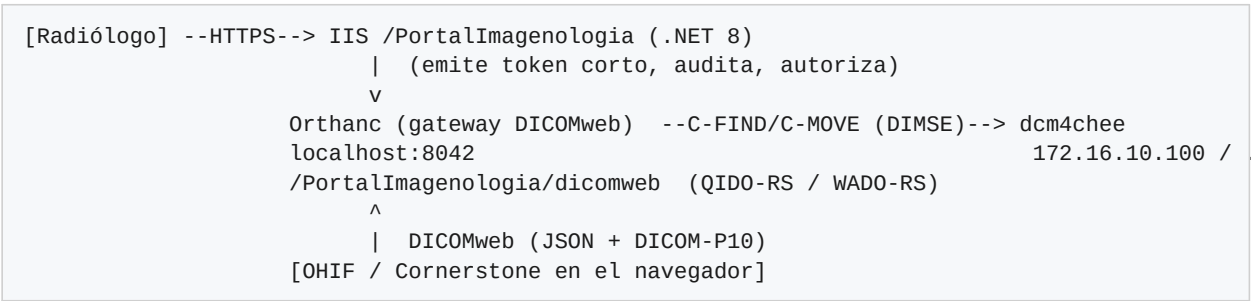
Corrección respecto al prompt inicial

El AE Title real es **DCM4CHEE** (no PACS_SERVER) y el WADO está en el puerto **8080**, contexto /wado. Los valores tipo PACS_SERVER o wadoUrl=http://172.16.10 eran placeholders.

3. Arquitectura de la solución (Plan A y Plan B)

Plan A — camino principal: Orthanc como pasarela DICOMweb + OHIF

Orthanc se instala como pasarela: consulta el PACS clásico por DIMSE (C-FIND/C-MOVE), cachea el estudio y lo re-expone como **DICOMweb moderno**. El visor (OHIF/Cornerstone) consume DICOMweb; la app .NET actúa como broker de seguridad (resuelve caso→StudyInstanceUID, emite token corto, audita).



Plan B — contingencia: visor 2D sobre WADO-URI JPEG de dcm4chee

Si Orthanc no está desplegado a tiempo, un visor 2D ligero consume el **WADO-URI JPEG** que dcm4chee ya expone (el mismo de Oviyam). Es de **fidelidad reducida** (sin window/level real ni MPR) pero no bloquea la entrega. Queda como contingencia explícita, no como diseño paralelo.

Aspecto	Plan A · DICOMweb + OHIF (nativo)	Plan B · WADO-URI JPEG 2D
Qué llega al navegador	DICOM crudo (16 bits, metadata)	Foto JPEG (8 bits, revelado fijo)
Window/Level dinámico	Sí, real	No
MPR (multiplanar)	Sí	No
Mediciones	Precisas (escala física)	Limitadas
Peso / dependencia	Mayor; requiere Orthanc	Muy ligero; ya disponible
Apto lectura diagnóstica	Sí	Solo vista de referencia

Decisión del PLAN-002 (aprobado)

Se elige **Plan A (Orthanc/OHIF)** como camino principal; **Plan B** queda documentado como contingencia si Orthanc no puede desplegarse/configurarse a tiempo. El despliegue de Orthanc se trata como precondition dura con evidencia real (C-ECHO/C-FIND).

4. Instalación y configuración de Orthanc (Windows Server 2012 R2)

4.1 Prerrequisitos en el servidor

- Windows Server 2012 R2 x64, con permisos de administrador.
- Microsoft Visual C++ Redistributable x64 (el que pida el instalador de Orthanc; habitualmente 2015–2022 x64).
- Espacio en disco para la caché de Orthanc (dimensionar según volumen; guía usa ~20 GB).
- Puertos libres: 8042/TCP (HTTP REST/DICOMweb, solo localhost) y 4242/TCP (DICOM SCP).

Compatibilidad de versión en 2012 R2

2012 R2 es un SO antiguo. Instala la versión estable de Orthanc para Windows y **verifica que el servicio arranca**. Si una build muy reciente no inicia (dependencia de runtime/OS), usa una versión LTS previa de Orthanc. Valida siempre contra la máquina real antes de dar por cerrado el paso.

4.2 Instalar Orthanc

- Descarga el **instalador oficial de Windows** (Orthanc Team / Osimis). Trae los plugins **DICOMweb** y **Stone Web Viewer**.
- Instálalo; queda como **servicio de Windows**. Ruta típica de plugins: C:/Program Files/Orthanc Server/Plugins.
- Crea las carpetas de datos: C:/Orthanc/Storage y C:/Orthanc/Index.

4.3 Configurar orthanc.json

Reemplaza (o fusiona) el `orthanc.json` de la instalación con esta configuración (comentada). Ajusta rutas, AET y, según el PACS real, el **AET del modality**:

```
{
  "Name": "Esculapio DICOMweb Gateway",
  "StorageDirectory": "C:/Orthanc/Storage",
  "IndexDirectory": "C:/Orthanc/Index",
  "MaximumStorageSize": 20000,          // ~20 GB de cache
  "MaximumStorageMode": "Recycle",      // recicla lo mas viejo al llenarse

  "HttpServerEnabled": true,
  "HttpPort": 8042,
  "SslEnabled": false,                  // el TLS lo termina IIS
  "RemoteAccessAllowed": false,        // *** solo localhost: lo alcanza IIS/ARR ***
  "AuthenticationEnabled": false,      // seguro porque solo escucha en localhost

  "DicomServerEnabled": true,
  "DicomAet": "ESCUAPIO_ORTHANC",      // *** registrar este AET EN el PACS (host+4242) ***
  "DicomPort": 4242,
  "DefaultEncoding": "Latin1",         // acentos legados; "Utf8" si el PACS lo emite
  "DicomAlwaysAllowEcho": true,
  "DicomAlwaysAllowStore": false,
  "DicomAlwaysAllowFind": false,
  "DicomAlwaysAllowMove": false,
  "DicomCheckModalityHost": true,

  "DicomModalities": {
    "pacs": {
      "AET": "DCM4CHEE",                // AE REAL del PACS (confirmado por HAR)
      "Host": "172.16.10.100",
      "Port": 11112,
      "AllowEcho": true, "AllowFind": true, "AllowMove": true, "AllowGet": true, "AllowStore": true
    }
  },

  "Plugins": [ "C:/Program Files/Orthanc Server/Plugins" ],

  "DicomWeb": {
    "Enable": true,
    "Root": "/PortalImagenologia/dicomweb/", // MISMO sub-path publico del proxy
    "EnableWado": true,
    "WadoRoot": "/PortalImagenologia/wado",
    "StudiesMetadata": "Full",            // OHIF necesita metadata completa
    "SeriesMetadata": "Full",
    "QidoCaseSensitive": false
  },

  "StoneWebViewer": { "DateFormat": "DD/MM/YYYY", "ShowInfoPanelAtStartup": "Always" }
}
```

Reinicia el servicio de Windows de Orthanc para aplicar la configuración.

El AET del PACS debe ser el REAL

En el andamiaje aparecía PACS_SERVER, pero el HAR confirmó que el AE real de dcm4chee es **DCM4CHEE**. Usa el valor real o el C-FIND/C-MOVE fallará.

4.4 Registro cruzado PACS ■ Orthanc (el paso que más se olvida)

Orthanc **no** reexpone el PACS automáticamente: su QIDO solo ve lo que tiene en caché. El flujo es C-FIND (localizar) → C-MOVE (traer a la caché) → recién ahí OHIF lo lee por DICOMweb. Para que el **C-MOVE** funcione, el PACS abre una asociación C-STORE de vuelta hacia Orthanc, así que:

- **En dcm4chee:** registrar a Orthanc como nodo/destino — AET ESCULAPIO_ORTHANC, host = IP del servidor de Orthanc, puerto **4242**.
- **En Orthanc:** el PACS ya está declarado en `DicomModalities.pacs`.

4.5 Firewall

- Orthanc → PACS en **11112/TCP** (C-FIND/C-MOVE saliente).
- PACS → Orthanc en **4242/TCP** (C-STORE de retorno del C-MOVE).

4.6 Pruebas de conectividad DICOM

Desde la REST de Orthanc (en el servidor):

```
# C-ECHO al PACS (debe responder 200)
POST http://localhost:8042/modalities/pacs/echo

# C-FIND por AccessionNumber real
POST http://localhost:8042/modalities/pacs/query
{ "Level": "Study", "Query": { "AccessionNumber": "<numero_real>" } }
```

Orden de pruebas sugerido

1) `http://localhost:8042` responde. 2) C-ECHO → 200. 3) C-FIND con un AccessionNumber real devuelve el estudio. 4) Abrir con motor **Stone** (sin build) valida el C-MOVE punta a punta. 5) Luego desplegar OHIF.

5. IIS: reverse proxy ARR + URL Rewrite

Orthanc escucha solo en `localhost:8042`; nadie de la red lo alcanza directo. IIS publica el DICOMweb bajo el mismo dominio HTTPS y sub-path del portal, terminando TLS y aplicando reglas de solo-lectura.

5.1 Requisitos IIS

- Instalar **Application Request Routing (ARR)** y **URL Rewrite** en IIS.
- En ARR: habilitar **Enable proxy** (Server Farms / ARR settings).

5.2 Estructura de aplicaciones anidadas

```
/PortalImagenologia      -> app ASP.NET Core (portal .NET 8)
/PortalImagenologia/ohif  -> Application estatica (OHIF)
/PortalImagenologia/dicomweb -> regla ARR: reverse proxy hacia http://localhost:8042/...
/PortalImagenologia/wado  -> (idem, si se usa WADO-URI)
```

5.3 Regla de reescritura (solo-lectura hacia Orthanc)

La regla endurecida reenvía GET a Orthanc y **rechaza POST/PUT/DELETE/PATCH** hacia /dicomweb (bloquea STOW). En URL Rewrite hay que registrar las *server variables* HTTP_X_FORWARDED_PROTO y HTTP_X_FORWARDED_HOST (y HTTP_AUTHORIZATION si se activa auth) en **View Server Variables**.

```
<!-- web.config del portal: reenvío de /dicomweb a Orthanc en localhost -->
<rule name="dicomweb-proxy" stopProcessing="true">
  <match url="^dicomweb/(.*)" />
  <conditions>
    <add input="{REQUEST_METHOD}" pattern="^(GET|HEAD|OPTIONS)$" />
  </conditions>
  <action type="Rewrite" url="http://localhost:8042/PortalImagenologia/dicomweb/{R:1}" />
  <serverVariables>
    <set name="HTTP_X_FORWARDED_PROTO" value="https" />
    <set name="HTTP_X_FORWARDED_HOST" value="{HTTP_HOST}" />
  </serverVariables>
</rule>
```

Gotcha de herencia

El web.config del portal padre debe envolver su <system.webServer> en <location path="." inheritInChildApplications="false">. Si no, el handler de ASP.NET Core y las reglas ARR se heredan en la app hija (OHIF) y rompen el servido estático.

6. OHIF: build y despliegue como aplicación estática

OHIF es una SPA en React: se **compila una vez** en una PC de desarrollo (Node LTS 18/20 + Yarn) y se publican los estáticos bajo /PortalImagenologia/ohif. **El Windows Server 2012 R2 no compila OHIF**; solo sirve los estáticos.

6.1 Build (en PC de desarrollo)

```
git clone https://github.com/OHIF/Viewers.git
cd Viewers
git checkout v3.11.0           # release v3 estable probada
yarn install
# copiar esculapio.js -> platform/app/public/config/esculapio.js
# Windows PowerShell:
$env:PUBLIC_URL="/PortalImagenologia/ohif/"; $env:APP_CONFIG="config/esculapio.js"; yarn build
```

Resultado en platform/app/dist/. Verifica que dist/app-config.js tenga routerBasename: "/PortalImagenologia/ohif" y que el data source apunte a /PortalImagenologia/dicomweb.

6.2 Publicar en el servidor

- Copiar todo platform/app/dist/ a C:\inetpub\...\PortalImagenologia\ohif\.
- Colocar el web.config de OHIF (MIME .wasm + fallback SPA) en la raíz de ohif\.
- En IIS Manager: clic derecho sobre ohif → **Convert to Application**, con App Pool propio **"No Managed Code"** (es estático).

Si carga la UI pero no las imágenes

Casi siempre es: (a) el MIME de .wasm (lo cubre el web.config de OHIF), (b) el DicomWeb.Root de Orthanc no coincide con /PortalImagenologia/dicomweb, o (c) el estudio aún no llegó por C-MOVE (revisa el job en Orthanc).

7. Integración con la aplicación .NET 8

La app .NET es el **broker**: autentica al radiólogo, resuelve el estudio y emite un token corto para que el visor acceda al DICOMweb. Archivos de referencia (andamiaje en ActualizacionCodigo/, no versionado):

Archivo	Rol
OrthancGatewayService.cs	C-FIND (buscar en PACS) y C-MOVE (traer a Orthanc) vía REST de Orthanc
VisorController.cs	Endpoints del visor: resolver caso→estudio, abrir visor, emitir token
VisorTokenService.cs	Token corto (JWT ~10 min) para acceso temporal al estudio
VisorAuditoriaService.cs	Trazabilidad: quién abrió qué estudio y cuándo
appsettings.Visor.json	Config: OrthancRestBaseUrl, OrthancDicomWebBaseUrl, OrthancAet, TokenMinutos

7.1 Configuración (appsettings — sección Visor)

```
"Visor": {
  "OrthancRestBaseUrl": "http://localhost:8042",
  "OrthancDicomWebBaseUrl": "http://localhost:8042/PortalImagenologia/dicomweb",
  "OrthancAet": "ESCALAPIO_ORTHANC",
  "PacsModalityName": "pacs",
  "TokenSecret": "<NO subir al repo: User-Secrets / variable de entorno>",
  "TokenMinutos": 10
}
```

7.2 Flujo de búsqueda (dos llaves de acceso)

```
Caso/Cuenta  --+
               +-> [.NET] resuelve -> PatientID / StudyInstanceUID
Identificacion-+
               |
               v
           C-FIND (Orthanc->PACS) -> C-MOVE a cache Orthanc
               |
               v
           OHIF abre /ohif/viewer?StudyInstanceUIDs={UID}
```

La app ya tiene la relación clínica caso↔paciente (vía API Esculapio); el visor solo necesita el **StudyInstanceUID** para el QIDO/WADO-RS. Conectar la pasarela al VisorController según gateway-wiring.txt (Resolver usa C-FIND; Abrir dispara C-MOVE).

8. Seguridad y hardening

- **HTTPS/TLS**: lo termina IIS; Orthanc con SslEnabled: false.

- **Aislamiento de Orthanc:** `RemoteAccessAllowed: false` → solo 127.0.0.1. Lo alcanzan únicamente IIS/ARR y el backend en la misma máquina.
- **Solo-lectura:** el proxy endurecido rechaza POST/PUT/DELETE/PATCH hacia /dicomweb (bloquea STOW). Opcional: `orthanc-readonly.lua` para reforzarlo en Orthanc.
- **DICOM entrante restringido:** `DicomAlwaysAllowStore/Find/Move: false + DicomCheckModalityHost: true` → solo la modalidad pacs declarada opera.
- **Autenticación:** la app .NET autentica al radiólogo (cookie + roles Administrador/Radiólogo). El visor recibe un token corto (JWT ~10 min).
- **Autorización por estudio:** el token se emite solo tras validar que el usuario puede ver ese caso.
- **Auditoría / trazabilidad:** `VisorAuditoriaService` registra accesos a estudios.
- **PHI:** nunca subir HAR, capturas con datos de pacientes ni credenciales a GitHub.

Defensa en profundidad opcional

Si algún día se expone Orthanc fuera de localhost: `AuthenticationEnabled: true + RegisteredUsers`, e inyectar el header `Authorization` en la regla ARR (más `OrthancUser/OrthancPassword` en `appsettings`).

9. Plan de pruebas end-to-end

- Servicio Orthanc arriba → `http://localhost:8042` responde en el servidor.
- C-ECHO al PACS → 200 (`POST /modalities/pacs/echo`).
- C-FIND con un `AccessionNumber` real → devuelve el estudio.
- Motor **Stone** (sin build): abrir un estudio → Orthanc hace C-MOVE y Stone lo muestra (valida la pasarela).
- IIS/ARR: `https://appsintranet.esculapiosis.com/PortalImagenologia/dicomweb/studies` responde (QIDO) por HTTPS.
- OHIF: navegar a `/PortalImagenologia/ohif/` carga la SPA; desde una grilla, "Ver imágenes" abre el estudio con branding Esculapio.
- Búsqueda por Caso/Cuenta y por Identificación → hasta visualizar el estudio.
- Auditoría: verificar que quedó el registro del acceso.

10. Bitácora de operación del enjambre (esta sesión)

Registro de lo realizado para que el **Avengers Swarm** produzca el diseño/plan del visor (operado por SSH en `swarm@157.173.123.197`).

10.1 Documentación y prompt

- Se creó `docs/PACS-exposicion.md` con los datos reales del PACS (extraídos del HAR).
- Se creó `docs/PROMPT-visor-diagnostico.md` (prompt maestro que reemplaza planes previos).

- Commits en WILSP1971/webImagenologia (rama main).

10.2 Reset del plan anterior y relanzamiento

- Se archivó PLAN-001 en ~/proyectos/.swarm_archive_20260807_reset/ y se limpió el estado vivo del .swarm.
- git pull en el VPS para bajar los docs nuevos.
- Se relanzó la tarea en sesión tmux persistente tarea_webImagenologia.

10.3 Fix del bloqueo de Bash (causa de que la 1ª corrida muriera)

Causa raíz

En ~/.claude/settings.json del enjambre existía "ask": ["Bash"], que forzaba confirmación de cada Bash; en modo headless (claude -p) nadie la responde y la tarea muere. Anulaba el --dangerously-skip-permissions.

- Se quitó "ask": ["Bash"] (backup en settings.json.bak_20260807); defaultMode queda en bypassPermissions.
- Se copió el andamiaje a ActualizacionCodigo/ en el VPS sin el HAR ni F12DOM (PHI) ni Instaladores/ (193 MB), y se añadió a .gitignore.

10.4 Resultado: PLAN-002 aprobado

El enjambre generó el **PLAN-002** (.swarm/PLAN-002.md) con Plan A (Orthanc/OHIF) como camino principal y Plan B (WADO-URI JPEG 2D) como contingencia. Quedó aprobado.

10.5 Comandos útiles de operación

```
# Ver/lanzar tarea (headless + failover)
ssh swarm@157.173.123.197 'bash ~/sistema-agentico/scripts/swtarea.sh webImagenologia "...'"

# Modo interactivo (TUI en vivo) en tmux persistente
tmux switch-client -t tarea_webImagenologia # si ya estas dentro de tmux
unset TMUX; tmux attach -t tarea_webImagenologia
# Salir sin cortar: Ctrl+b luego d

# Seguir avance sin entrar
git -C ~/proyectos/webImagenologia log --oneline -10
cat ~/proyectos/.swarm/CHECKPOINTS.md
```

11. Solución de problemas y puesta en marcha de Orthanc

11.1 Error 1359 al detener/reiniciar el servicio Orthanc

"Windows no pudo detener el servicio Orthanc. Error 1359: Error interno" es un fallo del control de servicios (SCM), **no** del orthanc.json (la config solo se lee al arrancar). Fuérralo por línea de comandos (CMD/PowerShell **como Administrador** en el servidor):

```
net stop Orthanc
REM si vuelve a fallar con 1359, mata el proceso y arranca:
taskkill /F /IM Orthanc.exe
net start Orthanc
```

11.2 Validar el orthanc.json en modo consola (paso clave)

Si Orthanc no vuelve a arrancar, corre el binario a mano para ver el error real (ajusta rutas):

```
"C:\Program Files\Orthanc Server\Orthanc.exe" --verbose "C:\Program Files\Orthanc Server\Configuration"
```

Si ves HTTP server listening on port 8042, el JSON es válido. Causas típicas de fallo:

- **Faltan las carpetas** C:\Orthanc\Storage / C:\Orthanc\Index → créalas (mkdir).
- **Claves duplicadas:** la instalación lee TODOS los *.json de Configuration\ y los fusiona; una clave repetida en dos archivos rompe el arranque. Deja un solo archivo con estas claves.
- **Ruta de Plugins incorrecta:** verifica que exista C:\Program Files\Orthanc Server\Plugins con OrthancDicomWeb.dll y OrthancStoneWebViewer.dll.

Para ver qué config usa el servicio: sc.exe qc Orthanc (¡con **.exe!** en PowerShell sc es alias de Set-Content) → mira el BINARY_PATH_NAME. En la instalación real: C:\Program Files\Orthanc Server\OrthancService.exe, y lee TODOS los *.json de Configuration\.

Error real observado: SQLite Unable to open the database (code 1002)

Orthanc arranca, carga plugins y luego muere con [SQLite: Unable to open the database].
Causa: **el directorio de la base de datos no existe**. El instalador de Windows deja por defecto StorageDirectory e IndexDirectory en C:\Orthanc. Solución: crear la carpeta y arrancar.

```
New-Item -ItemType Directory -Force -Path C:\Orthanc | Out-Null
Start-Service Orthanc
Get-Service Orthanc           # debe quedar Running; probar http://localhost:8042
```

11.2b Aplicar la config del gateway con un archivo de override (recomendado)

La carpeta Configuration\ del instalador trae ~15 archivos .json que Orthanc **fusiona** (el último en orden alfabético gana). En vez de reescribir el orthanc.json de 42 KB, se agrega zz-esculapio-gateway.json (el prefijo zz- asegura que cargue al final) con SOLO nuestras claves:

```
{
  "DicomAet": "ESCALAPIO_ORTHANC",
  "DicomPort": 4242,
  "RemoteAccessAllowed": false,
  "DicomModalities": {
    "pacs": { "AET": "DCM4CHEE", "Host": "172.16.10.100", "Port": 11112,
      "AllowEcho": true, "AllowFind": true, "AllowMove": true, "AllowGet": true, "AllowStore": true
    },
    "DicomWeb": { "Enable": true, "Root": "/PortalImagenologia/dicomweb/",
      "EnableWado": true, "WadoRoot": "/PortalImagenologia/wado",
      "StudiesMetadata": "Full", "SeriesMetadata": "Full" }
  },
}
```

Colocar en C:\Program Files\Orthanc Server\Configuration\, Restart-Service Orthanc, y verificar:

```
Invoke-RestMethod http://localhost:8042/system | Select Name,DicomAet,Version
Invoke-RestMethod -Method Post http://localhost:8042/modalities/pacs/echo
```

OHIF ya viene integrado en Orthanc 1.12.1

El log de arranque muestra Registering plugin 'ohif' (version 1.0) sirviendo en /ohif/, además de DICOMweb 1.15 y Stone Web Viewer 2.5. Es muy probable que **no haga falta compilar OHIF por separado**: Orthanc lo sirve directamente. Se valida abriendo un estudio ya en caché por /ohif/.

11.3 Acceso al PACS para registrar ESCULAPIO_ORTHANC (para C-MOVE)

dcm4chee corre en otro servidor (172.16.10.100). Se administra por navegador; según la versión:

- **dcm4chee-arc 5.x**: <http://172.16.10.100:8080/dcm4chee-arc/ui2> → Configuration → Devices/AE → nuevo Network AE.
- **dcm4chee 2.x**: <http://172.16.10.100:8080/dcm4chee-web3> → AE Management → New AET.

Registrar: AET ESCULAPIO_ORTHANC, Host = IP del servidor web (donde corre Orthanc), Port **4242**. Requiere credenciales de administrador del PACS.

Atajo: C-GET evita tocar el PACS

Si dcm4chee soporta **C-GET**, Orthanc recupera por la misma conexión saliente y NO hace falta registrar su AE en el PACS (config ya trae AllowGet: true). Pruébalo antes de depender del admin del PACS. C-ECHO (POST /modalities/pacs/echo) no requiere registro y valida conectividad.

11.4 Herramientas a descargar (servidor web)

Herramienta	Dónde	Archivo
Orthanc Windows (plugins DICOMweb+StoneWebViewer)	orthanc.uclouvain.be/downloads/windows-64	instalador .exe
IIS URL Rewrite 2.1	iis.net/downloads/microsoft/url-rewrite	rewrite_amd64_en-US.msi
IIS ARR 3.0	iis.net/downloads/microsoft/application-request-routing	requestRouter_amd64.msi

Herramienta	Dónde	Archivo
OHIF Viewer (compilar en PC dev)	github.com/OHIF/Viewers (v3.11.0)	—

Los archivos de configuración (orthanc.json con AE real, reglas IIS/ARR, config y web.config de OHIF) se entregan en el paquete PortalImagenologia-config.zip con su LEEME-INSTALACION.txt.

Anexo A. Datos reales de referencia

Parámetro	Valor real
PACS dcm4chee — AE Title	DCM4CHEE
dcm4chee — hosts	172.16.10.100 (Campbell/Fundación), 172.16.50.100 (Santa Marta)
dcm4chee — puerto DIMSE	11112
dcm4chee — WADO-URI	http://host:8080/wado (imageType JPEG)
Orthanc — AE Title	ESCALAPIO_ORTHANC
Orthanc — DICOM SCP	192.168.2.17:4242
Orthanc — HTTP/REST/DICOMweb	localhost:8042
DICOMweb Root (Orthanc e IIS)	/PortalImagenologia/dicomweb/
OHIF (estáticos IIS)	/PortalImagenologia/ohif
Portal .NET 8	https://appsintranet.esculapiosis.com/PortalImagenologia
Repo	https://github.com/WILSP1971/webImagenologia

Recordatorio PHI

Los IPs y AE Titles son datos de infraestructura interna. **Nunca** incluir en el repositorio archivos HAR, capturas o exportes con nombres/identificaciones de pacientes.